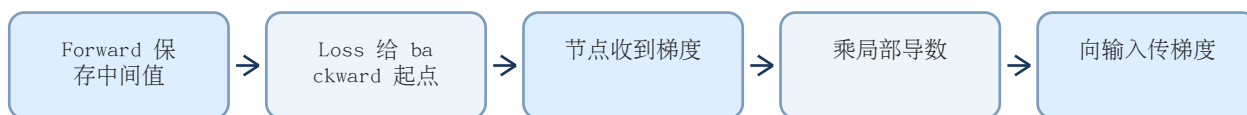


第23讲核心概念速查卡

Computational Graph 与节点级 Backward

核心概念：速查卡

本讲把 Chain Rule 变成真正可执行的 backward 规则。目标是：看到一个计算图节点时，知道它在 Forward 保存什么，在 Backward 收到什么梯度、乘哪个局部导数、把梯度传给哪些输入。



建议学习方式：先掌握四类标量节点规则 -> 再手工走完整图 -> 最后用数值差分检查每个参数梯度。

1. Backward 统一模板

传给输入的梯度 = 收到的梯度 $\times$ 局部导数

若 $z=f(a,b)$:

$$g_a = g_z \times dz/da$$

$$g_b = g_z \times dz/db$$

2. 四类节点规则

节点	Backward
$z=a+b$	$g_a=g_z, g_b=g_z$
$z=a \times b$	$g_a=g_z \times b, g_b=g_z \times a$
$z=a^2$	$g_a=g_z \times 2a$
$z=\text{ReLU}(a)$	$g_a=g_z \times 1(a>0)$

3. 完整例子答案

$x=2, w=3, b=-1, \text{target}=4$

$m=6 \rightarrow z=5 \rightarrow a=5 \rightarrow d=1 \rightarrow L=1$

$dL/dw=4, dL/db=2, dL/dx=6$

4. Seed gradient

$dL/dL = 1$

5. 多路径规则

$\text{grad}[x] = \text{sum}(\text{all path contributions})$

实现时直觉:

$\text{grad}[x] += \text{contribution}$

6. 三个必须区分

概念	含义
局部导数	当前节点自身的变化率
收到的梯度	Loss 对节点输出的敏感度
传出的梯度	Loss 对节点输入的敏感度

7. ReLU gate

input>0 -> gradient passes

input<0 -> gradient becomes 0

8. PyTorch 连接

`loss.backward()`: 自动执行反向图遍历

`parameter.grad`: 保存参数梯度

`optimizer.step()`: 使用梯度更新参数

共享参数 / 多路径: 梯度需要累加